

In Java 8, the terms **function** and **method** are closely related, but they are **not the same**. Java is an object-oriented language, so everything executable is ultimately a **method**. Java 8 introduced **functional programming** concepts (lambdas and functional interfaces), which brought the idea of **functions** into Java.

Difference Between Function and Method

Feature	Function	Method
Definition	A block of behavior represented as an object (usually through a lambda expression or method reference).	A block of code declared inside a class or interface.
Exists Independently?	✗ No. In Java, a function is represented by a functional interface.	✓ Yes. Every method belongs to a class or interface.
Introduced	Functional programming support in Java 8	Present since Java 1.0
Declaration	Lambda expression or method reference	Using a method declaration
Invocation	Through a functional interface (apply(), test(), accept(), etc.)	By calling the method directly
Can be passed as data?	✓ Yes	✗ Not directly (unless wrapped as a method reference or lambda)
Stored in variables?	✓ Yes	✗ No

1. What is a Method?

Definition

A **method** is a named block of code defined inside a class or interface that performs a specific task.

Syntax

```
returnType methodName(parameters) {  
    // code
```

```
}
```

Example

```
public class Calculator {  
  
    public int add(int a, int b) {  
        return a + b;  
    }  
  
}
```

Usage

```
Calculator c = new Calculator();  
  
System.out.println(c.add(10,20));
```

Output

30

Here:

- `add()` is a **method**.
 - It belongs to the Calculator class.
-

2. What is a Function in Java 8?

Definition

A **function** is behavior represented as an object using a **functional interface**.

Example

```
import java.util.function.Function;  
  
Function<Integer,Integer> square =  
    x -> x*x;  
  
System.out.println(square.apply(5));
```

Output

25

Here

`x -> x*x`

is the **function (lambda expression)**.

`Function<Integer,Integer>` is the **functional interface**.

Method vs Function Example

Method

```
class Demo {  
    static int square(int x) {  
        return x*x;  
    }  
}
```

Calling it

```
System.out.println(Demo.square(5));
```

Output

25

Function

```
Function<Integer,Integer> square =  
    x -> x*x;
```

```
System.out.println(square.apply(5));
```

Output

25

Both produce the same result, but they are invoked differently.

Can a Method Become a Function?

Yes, using a **method reference**.

Method

```
class Demo {  
    static int square(int x) {  
        return x*x;  
    }  
}
```

Convert it into a function

```
Function<Integer,Integer> fun =  
    Demo::square;  
  
System.out.println(fun.apply(5));
```

Output

25

The method is now being used as a function through the Function functional interface.

Functions are Objects

Methods are not objects.

Functions (represented by functional interfaces) are objects.

Example

```
Function<Integer,Integer> square =  
    x -> x*x;  
  
Function<Integer,Integer> cube =  
    x -> x*x*x;
```

Store in a list

```
List<Function<Integer,Integer>> list =  
Arrays.asList(square,cube);
```

Execute

```
for(Function<Integer,Integer> f : list){  
    System.out.println(f.apply(2));  
}
```

Output

```
4  
8
```

Methods cannot be stored like this directly.

Passing Functions as Parameters

Example

```
public static void execute(Function<Integer,Integer> f){  
    System.out.println(f.apply(10));  
}
```

Call

```
execute(x->x*x);  
execute(x->x+100);
```

Output

```
100  
110
```

Here, different functions are passed to the same method.

Passing Methods as Parameters

Directly passing a method is not allowed.

Instead, pass a **method reference**.

```
execute(Demo::square);
```

Function Composition

One major advantage of functions is that they can be combined.

Example

```
Function<Integer,Integer> square =  
x->x*x;
```

```
Function<Integer,Integer> add10 =  
x->x+10;
```

```
Function<Integer,Integer> result =  
square.andThen(add10);
```

```
System.out.println(result.apply(5));
```

Output

35

Flow

```
5  
↓  
square  
25  
↓  
add10  
35
```

Methods cannot be composed this way without writing additional code.

Method References

Method

```
public static int square(int x){  
  
    return x*x;  
  
}
```

Function

```
Function<Integer,Integer> fun =
```

```
Demo::square;
```

Equivalent lambda

```
Function<Integer,Integer> fun =  
x->Demo.square(x);
```

Stream API Example

Using methods

```
public static boolean isEven(int n){  
    return n%2==0;  
}
```

Method reference

```
numbers.stream()  
    .filter(Demo::isEven)  
    .forEach(System.out::println);
```

Equivalent lambda

```
numbers.stream()  
    .filter(n->Demo.isEven(n))  
    .forEach(System.out::println);
```

Real-World Analogy

Imagine a restaurant.

Method

A chef has a recipe named **Make Pizza**.

You ask the chef directly:

```
Chef.makePizza()
```

The chef prepares the pizza.

Function

Now imagine the recipe itself is written on a recipe card.

You can:

- hand it to another chef,
- store it in a file,
- combine it with another recipe,
- pass it to someone else.

That recipe card is similar to a **function object** in Java.

Summary

Feature	Method	Function
Belongs to	Class or interface	Functional interface instance
Introduced	Java 1.0	Java 8 (functional programming support)
Can be stored in a variable	✗ No	✓ Yes
Can be passed as an argument	✗ Not directly	✓ Yes
Can be returned from another method	✗ Not directly	✓ Yes
Supports composition	✗ No	✓ Yes (andThen(), compose())
Can use lambda syntax	✗ No	✓ Yes
Can use method references	Can be referenced	Represents the target of the reference

Key Takeaways

- A **method** is a named block of code defined in a class or interface.
- A **function** in Java 8 is behavior represented as an object through a **functional interface**, usually implemented with a **lambda expression** or **method reference**.
- Methods perform the work; functions allow that behavior to be stored, passed around, composed, and executed later, enabling Java's functional programming features such as the Stream API.

The short answer is:

- No, Java does not automatically convert a function (lambda) into a method.
- Yes, you can manually move the logic from a lambda into a method and then refer to that method using a method reference (::).

This is the opposite direction of what Java usually does. Java commonly converts a **method into a function** using a method reference, not a function into a method.

Understanding the Difference

Method

A **method** is declared inside a class.

```
class Calculator {  
  
    public static int square(int x) {  
        return x * x;  
    }  
  
}
```

Call it directly:

```
System.out.println(Calculator.square(5));
```

Output

25

Function (Lambda)

A function is represented by a **functional interface**.

```
Function<Integer, Integer> square =  
    x -> x * x;
```

```
System.out.println(square.apply(5));
```

Output

25

Can Java Automatically Convert This Lambda into a Method?

```
Function<Integer,Integer> square =  
    x -> x*x;
```

Can Java automatically create

```
public static int square(int x){  
    return x*x;  
}
```

Answer: No.

The lambda expression exists as an implementation of the Function interface. The compiler generates implementation details (often using invokedynamic and hidden implementation classes), but it **does not create a named Java method that you can call directly**.

Manual Conversion (Recommended)

Suppose you start with this lambda:

```
Function<Integer,Integer> square =  
    x -> x*x;
```

Move the logic into a method:

```
class Demo {  
  
    public static int square(int x){  
  
        return x*x;  
  
    }  
  
}
```

Then use a method reference:

```
Function<Integer,Integer> fun =  
    Demo::square;  
  
System.out.println(fun.apply(5));
```

Output

25

Flow

Function



Demo::square



square()



25

Example 1 - Predicate

Lambda

```
Predicate<String> empty =  
    s -> s.isEmpty();
```

Convert to a method

```
class Demo {  
  
    public static boolean isEmpty(String s){  
  
        return s.isEmpty();  
  
    }  
  
}
```

Use the method

```
Predicate<String> empty =  
    Demo::isEmpty;
```

Equivalent lambda

```
Predicate<String> empty =  
    s -> Demo.isEmpty(s);
```

Example 2 - Consumer

Lambda

```
Consumer<String> printer =
```

```
s -> System.out.println(s);
```

Convert

```
class Demo {  
  
    public static void print(String s){  
  
        System.out.println(s);  
  
    }  
  
}
```

Method Reference

```
Consumer<String> printer =  
    Demo::print;
```

Example 3 – Supplier

Lambda

```
Supplier<Date> supplier =  
    () -> new Date();
```

Convert

```
class Factory {  
  
    public static Date createDate(){  
  
        return new Date();  
  
    }  
  
}
```

Method Reference

```
Supplier<Date> supplier =  
    Factory::createDate;
```

Example 4 – Function

Lambda

```
Function<String,Integer> length =  
    s -> s.length();
```

Convert

```
class Util {  
  
    public static int length(String s){  
  
        return s.length();  
  
    }  
  
}
```

Method Reference

```
Function<String,Integer> length =  
    Util::length;
```

Example 5 – BinaryOperator

Lambda

```
BinaryOperator<Integer> add =  
    (a,b)->a+b;
```

Convert

```
class MathUtil {  
  
    public static int add(int a,int b){  
  
        return a+b;  
  
    }  
  
}
```

Method Reference

```
BinaryOperator<Integer> add =  
    MathUtil::add;
```

Real-World Example

Imagine an employee salary calculation.

Lambda

```
Function<Employee,Double> salary =  
    emp -> emp.getSalary();
```

Convert

```
class EmployeeUtil {  
  
    public static double salary(Employee emp){  
  
        return emp.getSalary();  
  
    }  
  
}
```

Method Reference

```
Function<Employee,Double> salary =  
    EmployeeUtil::salary;
```

Why Convert a Lambda into a Method?

1. Code Reuse

Instead of writing

```
x -> x*x
```

multiple times,

write

```
MathUtil::square
```

2. Better Readability

Instead of

```
numbers.stream()  
    .map(x -> x*x)
```

write

```
numbers.stream()
    .map(MathUtil::square)
```

3. Easier Testing

Method

```
public static int square(int x){
    return x*x;
}
```

can be unit tested independently.

4. Cleaner Stream Code

Lambda

```
employees.stream()
    .filter(e -> e.isActive())
    .map(e -> e.getSalary())
```

Method Reference

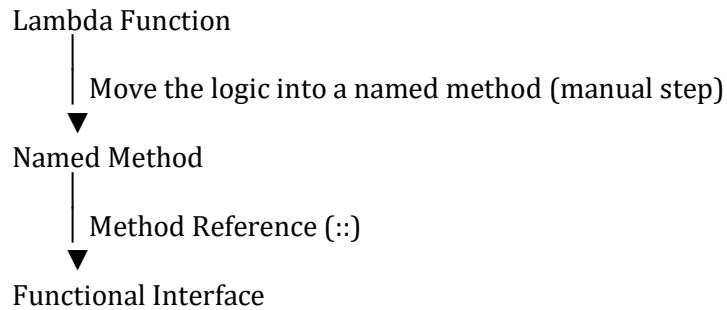
```
employees.stream()
    .filter(Employee::isActive)
    .map(Employee::getSalary)
```

Comparison

Lambda (Function)	Equivalent Method	Method Reference
x -> x*x	square(int x)	MathUtil::square
s -> s.length()	length(String s)	Util::length
s -> s.isEmpty()	isEmpty(String s)	Util::isEmpty
(a,b) -> a+b	add(int,int)	MathUtil::add
s -> System.out.println(s)	print(String s)	Demo::print
() -> new ArrayList<>()	createList()	Factory::createList

Function → Method → Function

You can think of the relationship like this:



Summary

Question	Answer
Can Java automatically convert a lambda function into a method?	✗ No
Can a developer manually move lambda logic into a method?	✓ Yes
Can that method then be used wherever a function is expected?	✓ Yes, by using a method reference (::)
Is this considered a best practice?	✓ Often yes, especially when the logic is reused or complex

The most common workflow in Java 8 is:

1. Write logic as a **named method**.
2. Pass that method as behavior using a **method reference (::)**.
3. The method reference is adapted by the compiler to the required **functional interface** (Function, Predicate, Consumer, Supplier, etc.). This gives you the readability and reusability of methods while still working seamlessly with lambdas and the Stream API.

Yes. **This is one of the main features introduced in Java 8.**

A method can be used as a function by assigning it to a **functional interface** using a **method reference (::)** or an equivalent **lambda expression**.

Technically, the method itself is not converted into a function. Instead, Java creates an implementation of a compatible **functional interface** that delegates to that method. This adaptation is done by the compiler and runtime.

How It Works

Suppose you have a normal method:

```
class MathUtil {  
  
    public static int square(int x) {  
        return x * x;  
    }  
  
}
```

Normally, you call it like this:

```
System.out.println(MathUtil.square(5));
```

Output

25

Convert the Method into a Function

```
import java.util.function.Function;
```

```
Function<Integer, Integer> square =  
    MathUtil::square;
```

```
System.out.println(square.apply(5));
```

Output

25

Flow

Method

MathUtil.square()

↓
▼
Method Reference (::)
↓
▼
Function<Integer,Integer>
↓
▼
apply(5)
↓
▼
25

Method Reference vs Lambda

Method

```
public static int square(int x) {  
    return x * x;  
}
```

Method Reference

```
Function<Integer,Integer> f =  
    MathUtil::square;
```

Equivalent Lambda

```
Function<Integer,Integer> f =  
    x -> MathUtil.square(x);
```

Both behave identically.

Example 1 - Function<T,R>

Method

```
class StringUtil {  
  
    public static int length(String s) {  
        return s.length();  
    }  
  
}
```

Convert

```
Function<String,Integer> length =  
    StringUtil::length;
```

Use

```
System.out.println(length.apply("Java"));
```

Output

4

Example 2 – Predicate

Method

```
class Validator {  
  
    public static boolean isAdult(int age) {  
        return age >= 18;  
    }  
  
}
```

Convert

```
Predicate<Integer> adult =  
    Validator::isAdult;
```

Use

```
System.out.println(adult.test(20));  
System.out.println(adult.test(15));
```

Output

true
false

Example 3 – Consumer

Method

```
class Printer {
```

```
    public static void print(String s) {  
        System.out.println(s);  
    }  
}
```

Convert

```
Consumer<String> printer =  
    Printer::print;
```

Use

```
printer.accept("Hello");
```

Output

Hello

Example 4 – Supplier

Method

```
class Factory {  
  
    public static String company() {  
        return "OpenAI";  
    }  
}
```

Convert

```
Supplier<String> supplier =  
    Factory::company;
```

Use

```
System.out.println(supplier.get());
```

Output

OpenAI

Example 5 - BinaryOperator

Method

```
class MathUtil {  
    public static int add(int a,int b) {  
        return a+b;  
    }  
}
```

Convert

```
BinaryOperator<Integer> add =  
    MathUtil::add;
```

Use

```
System.out.println(add.apply(10,20));
```

Output

30

Example 6 - Comparator

Method

```
List<String> list =  
Arrays.asList("Banana","Apple","Orange");  
  
list.sort(String::compareToIgnoreCase);  
  
System.out.println(list);
```

Output

[Apple, Banana, Orange]

Equivalent lambda

```
list.sort((a,b) ->  
    a.compareToIgnoreCase(b));
```

Example 7 – Stream API

Method

```
class EmployeeUtil {  
  
    public static boolean isHighSalary(Employee e) {  
        return e.getSalary() > 50000;  
    }  
  
}
```

Convert

```
employees.stream()  
    .filter(EmployeeUtil::isHighSalary)  
    .forEach(System.out::println);
```

Equivalent lambda

```
employees.stream()  
    .filter(e ->  
        EmployeeUtil.isHighSalary(e))  
    .forEach(System.out::println);
```

Example 8 – Constructor as a Function

Constructor

```
ArrayList<String>::new
```

Assigned to a Supplier

```
Supplier<ArrayList<String>> supplier =  
    ArrayList::new;
```

```
ArrayList<String> list = supplier.get();
```

Equivalent lambda

```
Supplier<ArrayList<String>> supplier =  
    () -> new ArrayList<>();
```

Four Types of Method References

Type	Syntax	Functional Interface Example
Static Method	ClassName::staticMethod	Function<String,Integer> parser = Integer::parseInt;
Instance Method of a Particular Object	object::method	Supplier<String> input = scanner::nextLine;
Instance Method of an Arbitrary Object of a Particular Type	ClassName::instanceMethod	Function<String,Integer> len = String::length;
Constructor	ClassName::new	Supplier<List<String>> list = ArrayList::new;

Method Signature Must Match

The method's signature must be compatible with the functional interface's single abstract method.

Example

Method

```
public static int square(int x)
```

Functional Interface

```
Function<Integer,Integer>
```

Why?

Functional Interface	Required Method
Function<Integer,Integer>	Integer apply(Integer)
Actual method	int square(int)

The parameter and return types are compatible (primitive int is automatically boxed/unboxed to Integer), so Java allows the method reference.

Why Convert Methods into Functions?

1. Reuse Existing Code

Instead of writing

```
x -> x*x
```

everywhere,

write

```
MathUtil::square
```

2. Cleaner Stream Code

Lambda

```
numbers.stream()  
    .map(x -> MathUtil.square(x));
```

Method Reference

```
numbers.stream()  
    .map(MathUtil::square);
```

3. Pass Behavior as Data

```
public static void execute(  
    Function<Integer,Integer> function) {  
  
    System.out.println(function.apply(10));  
  
}
```

Call

```
execute(MathUtil::square);
```

4. Easier Testing

A named method can be tested independently and then reused through method references.

Method → Function Flow

Named Method



Method Reference (::)



Functional Interface



Lambda-like Function Object

Summary

Feature	Method	Function (via Functional Interface)
Declared in a class	✓ Yes	✗ No
Can be called directly	✓ Yes	Through apply(), test(), accept(), get(), etc.
Can be passed as an argument	✗ Not directly	✓ Yes
Can be stored in a variable	✗ No	✓ Yes
Can be referenced with ::	✓ Yes	Not applicable

Key Takeaways

- Java 8 **does not change the method into a different kind of language construct**. Instead, it **adapts** a compatible method to a **functional interface** using a **method reference** or an equivalent lambda.
- This allows existing methods to participate in functional programming, making them easy to pass to APIs such as the Stream API, Collections.sort(), CompletableFuture, and many other Java 8+ APIs.

The short answer is:

- **No, not in the way the question is phrased.**
- **A lambda expression is itself one way of writing a function** in Java 8. So a function is **not converted into a lambda**; rather, **a lambda is the implementation of a functional interface (the function).**

This distinction is important.

Relationship Between Functional Interface, Function, and Lambda

Functional Interface



Lambda Expression

implements



Function Object

For example:

```
Function<Integer, Integer> square = x -> x * x;
```

Here:

- `Function<Integer, Integer>` → **Functional interface**
- `x -> x * x` → **Lambda expression**
- `square` → **Function object** (an instance implementing Function)

So the lambda **creates** the function object.

What is a Lambda Expression?

A **lambda expression** is a concise way to implement the single abstract method of a functional interface.

Example:

```
Function<Integer,Integer> square =  
    x -> x * x;
```

Equivalent anonymous class:

```
Function<Integer,Integer> square =  
    new Function<Integer,Integer>() {  
        @Override  
        public Integer apply(Integer x) {  
            return x * x;  
        }  
    };
```

What is a Function?

In Java 8, when people say "function," they usually mean an object implementing a functional interface.

Example:

```
Function<String,Integer> length =  
    String::length;
```

or

```
Function<String,Integer> length =  
    s -> s.length();
```

Both produce the same kind of function object.

Lambda Creates the Function

Example

```
Function<Integer,Integer> square =  
    x -> x * x;
```

Flow

Lambda Expression

x -> x*x



Implements Function<Integer,Integer>



Function Object

Example 1 – Function

Lambda

```
Function<Integer,Integer> square =  
    x -> x*x;
```

Use

```
System.out.println(square.apply(5));
```

Output

25

Example 2 – Predicate

```
Predicate<String> empty =  
    s -> s.isEmpty();
```

Use

```
System.out.println(empty.test(""));
```

Output

true

Here,

```
s -> s.isEmpty()
```

is already the lambda expression that creates the Predicate.

Example 3 – Consumer

```
Consumer<String> printer =  
    s -> System.out.println(s);
```

Use

```
printer.accept("Hello");
```

Output

Hello

Example 4 - Supplier

```
Supplier<Double> random =  
    () -> Math.random();
```

Use

```
System.out.println(random.get());
```

Function Written Using Lambda

```
Function<String,Integer> length =  
    s -> s.length();
```

Equivalent using a method reference

```
Function<String,Integer> length =  
    String::length;
```

Both create the same type of function object.

Function Written Using an Anonymous Class

Before Java 8

```
Function<Integer,Integer> square =  
new Function<Integer,Integer>() {  
  
    @Override  
    public Integer apply(Integer x) {  
        return x*x;  
    }  
  
};
```

Java 8 Lambda

```
Function<Integer,Integer> square =  
    x -> x*x;
```

The lambda is simply shorter.

Can One Function Be Rewritten as Another Lambda?

Yes.

Example

Current function

```
Function<Integer,Integer> square =  
    x -> x*x;
```

Rewrite it

```
Function<Integer,Integer> square =  
    number -> number*number;
```

or

```
Function<Integer,Integer> square =  
    value -> value*value;
```

All are equivalent because only the parameter name changed.

Lambda vs Method Reference

Lambda

```
Function<String,Integer> length =  
    s -> s.length();
```

Method Reference

```
Function<String,Integer> length =  
    String::length;
```

Both represent the same function.

Lambda vs Anonymous Class

Anonymous Class

Runnable r =

```
new Runnable(){  
    @Override  
    public void run(){  
        System.out.println("Running");  
    }  
};
```

Lambda

```
Runnable r =  
    () -> System.out.println("Running");
```

Both create a Runnable.

Stream API Example

Lambda

```
List<String> names =  
Arrays.asList("Java","Python");  
  
names.stream()  
    .map(s -> s.length())  
    .forEach(System.out::println);
```

Method Reference

```
names.stream()  
    .map(String::length)  
    .forEach(System.out::println);
```

Real-World Analogy

Imagine a recipe.

Functional Interface = Recipe template

Recipe:

Take ingredients

Cook

Return food

Lambda Expression = The actual recipe you write

Take bread
Toast it
Add butter

Function Object = The completed recipe card that can now be passed around and used.

Common Misconception

Many developers say:

"Convert a function to a lambda."

More accurately:

- A **lambda expression implements a functional interface**.
 - The resulting object is often referred to as a **function**.
 - Therefore, a lambda is **one syntax for defining a function**, not something a function is converted into.
-

Summary Table

Concept	Definition	Example
Functional Interface	Interface with one abstract method	Function<T,R>
Lambda Expression	Concise implementation of that method	x -> x * x
Function Object	Instance implementing the functional interface	Function<Integer,Integer> square = x -> x * x;
Method Reference	Another concise implementation using an existing method	MathUtil::square

Key Takeaways

- A **function cannot be converted to a lambda** because a lambda is already **the expression used to create a function** in Java.
- If you already have a function object (for example, a Function, Predicate, or Consumer), it is **already backed by** a lambda, a method reference, or an anonymous class.

- The three common ways to implement a functional interface in Java 8 are:
 1. **Lambda expression** (most common)
 2. **Method reference** (::)
 3. **Anonymous class** (pre-Java 8 style)

So, the more precise statement is:

A lambda expression is a way to define a function in Java 8, not something that a function is converted into.

Yes, **conceptually**, but it's important to use the correct Java terminology.

A **lambda expression** by itself is just **syntax**. When you assign it to a **functional interface**, Java creates a **function object** (an instance of that functional interface). So, in practice, a lambda becomes a function object.

Relationship Between Lambda and Function

Lambda Expression



Assigned to a Functional Interface



Function Object

Example:

```
import java.util.function.Function;
```

```
Function<Integer, Integer> square = x -> x * x;
```

Here:

- `x -> x * x` → **Lambda expression**
 - `Function<Integer, Integer>` → **Functional interface**
 - `square` → **Function object** (an implementation of Function)
-

Step-by-Step Example

Step 1: Write a Lambda

```
x -> x * x
```

This is **only a lambda expression**. By itself, it has no type.

It cannot stand alone like this:

```
// Invalid  
x -> x * x;
```

Java needs to know **which functional interface** it should implement.

Step 2: Assign It to a Functional Interface

```
Function<Integer, Integer> square =  
    x -> x * x;
```

Now Java knows:

- Input type → Integer
- Return type → Integer
- Abstract method → apply()

Java creates an implementation of Function.

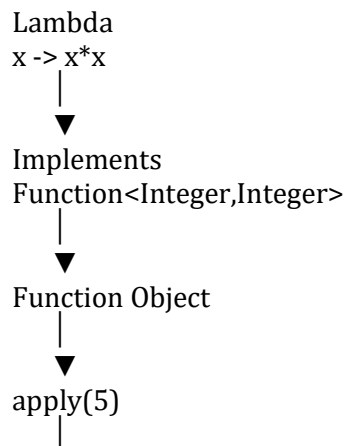
Step 3: Use It Like a Function

```
System.out.println(square.apply(5));
```

Output

25

What Happens Internally?



Example 1 – Predicate

Lambda

```
Predicate<String> empty =  
    s -> s.isEmpty();
```

Use

```
System.out.println(empty.test(""));
System.out.println(empty.test("Java"));
```

Output

```
true
false
```

Here:

- Lambda

```
s -> s.isEmpty()
```

becomes a Predicate<String>.

Example 2 – Consumer

Lambda

```
Consumer<String> printer =  
    s -> System.out.println(s);
```

Use

```
printer.accept("Hello");
```

Output

```
Hello
```

Example 3 – Supplier

Lambda

```
Supplier<Double> random =  
    () -> Math.random();
```

Use

```
System.out.println(random.get());
```

Possible Output

0.845732

Example 4 – BinaryOperator

Lambda

```
BinaryOperator<Integer> add =  
    (a,b) -> a + b;
```

Use

```
System.out.println(add.apply(10,20));
```

Output

30

Example 5 – Comparator

Lambda

```
Comparator<String> cmp =  
    (a,b) -> a.compareTo(b);
```

Use

```
List<String> list =  
Arrays.asList("C","A","B");
```

```
list.sort(cmp);
```

```
System.out.println(list);
```

Output

[A, B, C]

Lambda vs Anonymous Class

Lambda

```
Function<Integer,Integer> square =  
    x -> x*x;
```

Equivalent Anonymous Class

```
Function<Integer,Integer> square =  
new Function<Integer,Integer>() {  
  
    @Override  
    public Integer apply(Integer x) {  
        return x*x;  
    }  
  
};
```

Both produce the same kind of function object.

Lambda vs Method Reference

Lambda

```
Function<String,Integer> length =  
    s -> s.length();
```

Method Reference

```
Function<String,Integer> length =  
    String::length;
```

Both create a Function<String, Integer>.

Lambda in Stream API

```
List<Integer> numbers =  
Arrays.asList(1,2,3,4,5);  
  
numbers.stream()
```

```
.filter(x -> x % 2 == 0)
.map(x -> x * x)
.forEach(System.out::println);
```

Output

4
16

Internally:

Lambda	Functional Interface
<code>x -> x % 2 == 0</code>	<code>Predicate<Integer></code>
<code>x -> x * x</code>	<code>Function<Integer,Integer></code>
<code>System.out::println</code>	<code>Consumer<Integer></code>

Why Doesn't the Lambda Have a Type by Itself?

A lambda expression is **target-typed**. Its type comes from the context.

Example

```
Function<Integer,Integer> f =
    x -> x*x;
```

Here Java infers:

- parameter type = Integer
- return type = Integer

If you write only:

```
x -> x*x;
```

Java reports an error because there is **no target functional interface**.

Real-World Analogy

Imagine a job description.

Functional Interface

Job:
Take one number
Return one number

Lambda

Multiply by 2

When the lambda is assigned to the job description, it becomes an employee performing that job (the function object).

Summary Table

Concept	Definition	Example
Functional Interface	Interface with one abstract method	Function<T,R>
Lambda Expression	Short implementation of that method	x -> x * x
Function Object	Instance implementing the functional interface	Function<Integer,Integer> square = x -> x * x;
Method Reference	Existing method used instead of a lambda	MathUtil::square

Key Takeaways

- **Yes**, a lambda expression becomes a **function object** when it is assigned to a compatible **functional interface**.
- A lambda expression **does not have an independent type**. It gets its type from the target functional interface.
- In Java 8, a lambda can implement **any functional interface**, such as:
 - Function
 - Predicate
 - Consumer
 - Supplier
 - UnaryOperator
 - BinaryOperator
 - Comparator
 - Runnable
 - Callable
- The compiler and runtime handle the creation of the functional interface implementation behind the scenes, allowing you to use the lambda as a function object.

A **lambda expression** and an **anonymous class** can both be used to implement a **functional interface**, but they are **not the same**. Lambda expressions were introduced in **Java 8** to provide a shorter, cleaner, and more efficient way to represent behavior.

Definition of an Anonymous Class

An **anonymous class** is a class without a name that is declared and instantiated in a single expression. It can extend a class or implement an interface.

Example:

```
Runnable r = new Runnable() {  
    @Override  
    public void run() {  
        System.out.println("Running...");  
    }  
};
```

Definition of a Lambda Expression

A **lambda expression** is a concise way to implement the **single abstract method** of a **functional interface**.

Equivalent lambda:

```
Runnable r = () -> System.out.println("Running...");
```

Anonymous Class vs Lambda

Feature	Anonymous Class	Lambda Expression
Introduced	Java 1.1	Java 8
Purpose	Create an unnamed class	Implement a functional interface concisely
Syntax	Verbose	Concise
Creates a new class	✓ Yes (compiler generates an anonymous class)	✗ No separate anonymous class is created; the lambda is implemented via the JVM's lambda machinery (invokedynamic)

Requires new keyword	✓ Yes	✗ No
Can implement multiple abstract methods	✓ Yes (if the interface/class allows)	✗ No, only functional interfaces (one abstract method)
Has its own this	✓ Yes	✗ No, this refers to the enclosing instance
Has its own scope	✓ Yes	✗ No, shares the enclosing scope

Example 1 – Runnable

Anonymous Class

```
Runnable r = new Runnable() {
    @Override
    public void run() {
        System.out.println("Hello");
    }
};
```

```
r.run();
```

Output

Hello

Lambda

```
Runnable r = () -> System.out.println("Hello");
```

```
r.run();
```

Output

Hello

The result is the same, but the lambda is much shorter.

Example 2 – Comparator

Anonymous Class

```
Comparator<String> comparator =
    new Comparator<String>() {
        @Override
```

```
    public int compare(String a, String b) {  
        return a.compareTo(b);  
    }  
};
```

Lambda

```
Comparator<String> comparator =  
    (a, b) -> a.compareTo(b);
```

Example 3 – Function

Anonymous Class

```
Function<Integer, Integer> square =  
    new Function<Integer, Integer>() {  
        @Override  
        public Integer apply(Integer x) {  
            return x * x;  
        }  
    };  
  
System.out.println(square.apply(5));
```

Output

25

Lambda

```
Function<Integer, Integer> square =  
    x -> x * x;  
  
System.out.println(square.apply(5));
```

Output

25

Example 4 – Predicate

Anonymous Class

```
Predicate<Integer> even =  
    new Predicate<Integer>() {  
        @Override  
        public boolean test(Integer x) {  
            return x % 2 == 0;  
        }  
    };
```

```
}  
};
```

Lambda

```
Predicate<Integer> even =  
    x -> x % 2 == 0;
```

Example 5 – Consumer

Anonymous Class

```
Consumer<String> printer =  
    new Consumer<String>() {  
        @Override  
        public void accept(String s) {  
            System.out.println(s);  
        }  
    };  
};
```

Lambda

```
Consumer<String> printer =  
    System.out::println;
```

or

```
Consumer<String> printer =  
    s -> System.out.println(s);
```

Difference in this

This is one of the most important differences.

Anonymous Class

```
class Demo {  
  
    String name = "Outer";  
  
    void test() {  
  
        Runnable r = new Runnable() {  
  
            String name = "Inner";  
  
            @Override  
            public void run() {
```

```
        System.out.println(this.name);
    }

};

    r.run();
}

}
```

Output

Inner

Here, this refers to the anonymous class object.

Lambda

```
class Demo {

    String name = "Outer";

    void test() {

        Runnable r = () -> {
            System.out.println(this.name);
        };

        r.run();
    }

}
```

Output

Outer

Here, this refers to the enclosing Demo object.

Variable Scope

Anonymous Class

```
int x = 10;

Runnable r = new Runnable() {
```

```
int x = 20;

@Override
public void run() {
    System.out.println(x);
}

};
```

Output

20

The anonymous class introduces a new scope.

Lambda

```
int x = 10;

Runnable r = () -> {
    System.out.println(x);
};

r.run();
```

Output

10

A lambda shares the enclosing scope, so it cannot redeclare x.

Access to Local Variables

Both anonymous classes and lambdas can access **final or effectively final** local variables.

```
int number = 100;

Runnable r = () -> System.out.println(number);

r.run();
```

Output

100

If number is modified after being captured, the code will not compile.

Stream API Example

Anonymous Class

```
numbers.stream()
    .filter(new Predicate<Integer>() {
        @Override
        public boolean test(Integer n) {
            return n % 2 == 0;
        }
    })
    .forEach(System.out::println);
```

Lambda

```
numbers.stream()
    .filter(n -> n % 2 == 0)
    .forEach(System.out::println);
```

The lambda version is significantly more readable.

When to Use an Anonymous Class

Use an anonymous class when you need:

- To extend a concrete or abstract class.
- To override more than one method (for example, inherited default methods if needed).
- A separate object with its own this and its own fields.

Example:

```
Thread thread = new Thread(new Runnable() {
    @Override
    public void run() {
        System.out.println("Running...");
    }
});
```

When to Use a Lambda

Use a lambda when:

- The target is a **functional interface**.
- You only need to implement one abstract method.
- You want concise, readable code.
- You are using APIs such as the Stream API, CompletableFuture, or collection operations.

Example:

```
numbers.stream()
    .map(x -> x * x)
    .filter(x -> x > 10)
    .forEach(System.out::println);
```

Summary

Aspect	Anonymous Class	Lambda
Syntax	Verbose	Compact
Java Version	1.1+	8+
Requires new	Yes	No
Implements	Any interface or extends a class	Functional interface only
Separate class generated	Yes	No separate anonymous class
this	Refers to anonymous class instance	Refers to enclosing instance
Own scope	Yes	No
Stream API usage	Possible but verbose	Preferred

Key Takeaways

- An **anonymous class** is a real unnamed class with its own identity, fields, methods, and this.
- A **lambda expression** is a compact implementation of the single abstract method of a **functional interface**.
- Lambdas are generally preferred in Java 8+ for functional programming because they are shorter, easier to read, and integrate naturally with the Stream API and other functional APIs. Anonymous classes remain useful when you need features that lambdas intentionally do not provide, such as extending a class or maintaining a separate object identity.

What is a Method Reference in Java 8?

A **method reference** is a **shorthand notation for a lambda expression** that simply calls an existing method or constructor.

Instead of writing a lambda that only invokes an existing method, you can use the **:: (double colon) operator**.

For example,

Lambda:

```
Consumer<String> printer =  
    s -> System.out.println(s);
```

Method Reference:

```
Consumer<String> printer =  
    System.out::println;
```

Both are **functionally equivalent**.

Definition

A **method reference** is a compact syntax that refers to an **existing method or constructor**, allowing it to be used wherever a compatible **functional interface** is expected.

The referenced method must have a signature compatible with the **Single Abstract Method (SAM)** of the target functional interface.

Syntax

ClassName::staticMethod

objectReference::instanceMethod

ClassName::instanceMethod

ClassName::new

Why Were Method References Introduced?

Before Java 8

```
Collections.sort(list, new Comparator<String>() {  
  
    @Override  
    public int compare(String a, String b) {  
        return a.compareTo(b);  
    }  
  
});
```

Java 8 Lambda

```
Collections.sort(list,  
    (a,b)->a.compareTo(b));
```

Java 8 Method Reference

```
Collections.sort(list,  
    String::compareTo);
```

The code becomes shorter and more readable.

Four Types of Method References

Type	Syntax	Example
Static Method	ClassName::staticMethod	Integer::parseInt
Instance Method of Particular Object	object::method	scanner::nextLine
Instance Method of Arbitrary Object	ClassName::instanceMethod	String::length
Constructor Reference	ClassName::new	ArrayList::new

1. Static Method Reference

Method

```
class MathUtil {  
  
    public static int square(int x){
```

```
        return x*x;
    }
}
```

Method Reference

```
Function<Integer,Integer> square =
    MathUtil::square;

System.out.println(square.apply(5));
```

Equivalent Lambda

```
Function<Integer,Integer> square =
    x -> MathUtil.square(x);
```

Output

25

2. Instance Method of a Particular Object

```
Scanner scanner =
new Scanner(System.in);

Supplier<String> input =
    scanner::nextLine;
```

Equivalent Lambda

```
Supplier<String> input =
    () -> scanner.nextLine();
```

3. Instance Method of an Arbitrary Object

```
Function<String,Integer> length =
    String::length;
```

Equivalent Lambda

```
Function<String,Integer> length =
    s -> s.length();
```

Output

4

4. Constructor Reference

```
Supplier<List<String>> supplier =  
    ArrayList::new;
```

Equivalent Lambda

```
Supplier<List<String>> supplier =  
    () -> new ArrayList<>();
```

How Does a Method Reference Work Internally?

This is where Java 8 becomes interesting.

Many people think

`String::length`

directly calls the method.

It does not.

Internally Java performs several steps.

Step 1

Suppose we write

```
Function<String,Integer> fun =  
    String::length;
```

Step 2

Compiler Checks

The compiler first verifies

`Function<T,R>`

contains

R apply(T t);

For

Function<String,Integer>

the SAM becomes

Integer apply(String s);

Step 3

Compiler Looks at Referenced Method

String.length()

Signature

int length()

Comparison

Function

↓

apply(String)

↓

String.length()

↓

Compatible

Because

Input

String

Output

Integer (boxing from int)

matches

apply(String)

the compiler accepts it.

Step 4

Compiler Generates Lambda Metadata

Instead of generating

```
new Function() {  
    public Integer apply(String s){  
        return s.length();  
    }  
}
```

Java 8 generates an **invokedynamic** instruction in the bytecode.

At runtime, the JVM links this instruction using the **LambdaMetafactory**, which creates an implementation of the target functional interface that delegates to the referenced method.

This avoids eagerly generating a separate anonymous class and allows the JVM to optimize lambda creation and invocation.

Conceptually, it behaves like:

```
Function<String,Integer> fun =  
new Function<String,Integer>(){  
    @Override  
    public Integer apply(String s){  
        return s.length();  
    }  
};
```

Step 5

At Runtime

When you write

```
fun.apply("Java");
```

Execution

apply()

↓

String.length()

↓

4

Internal Flow

String::length

|

▼

Compiler checks

Function<String,Integer>

|

▼

Method Signature Matches

|

▼

invokedynamic + LambdaMetafactory

|

▼

Creates Function implementation

|

▼

apply()

|

▼

length()

|



Static Method Internal Flow

Example

```
Function<String,Integer> parser =  
    Integer::parseInt;
```

Equivalent Lambda

```
Function<String,Integer> parser =  
    s -> Integer.parseInt(s);
```

Execution

```
apply("123")
```

↓

```
Integer.parseInt("123")
```

↓

```
123
```

Consumer Example

```
Consumer<String> printer =  
    System.out::println;
```

Equivalent Lambda

```
Consumer<String> printer =  
    s -> System.out.println(s);
```

Execution

```
accept("Hello")
```

↓

```
println()
```

↓

Hello

Predicate Example

Method Reference

```
Predicate<String> empty =  
    String::isEmpty;
```

Equivalent Lambda

```
Predicate<String> empty =  
    s -> s.isEmpty();
```

Execution

```
test("")
```

↓

```
isEmpty()
```

↓

```
true
```

Supplier Example

Method

```
class Factory{  
  
    static Employee create(){  
  
        return new Employee();  
  
    }  
  
}
```

Method Reference

```
Supplier<Employee> supplier =  
    Factory::create;
```

Equivalent Lambda

```
Supplier<Employee> supplier =  
    () -> Factory.create();
```

Execution

```
get()
```

↓

```
Factory.create()
```

↓

```
Employee
```

Constructor Reference Internally

```
Supplier<List<String>> supplier =  
    ArrayList::new;
```

Equivalent

```
Supplier<List<String>> supplier =  
    () -> new ArrayList<>();
```

Execution

```
get()
```

↓

```
new ArrayList()
```

↓

```
Object Returned
```

Stream API Example

```
List<String> names =  
Arrays.asList("Java", "Python", "C");
```

```
names.stream()  
    .map(String::length)  
    .forEach(System.out::println);
```

Internally

Stream

↓

map()

↓

Function<String,Integer>

↓

apply()

↓

String.length()

↓

4

6

1

Why Use Method References?

Better Readability

Lambda

x -> Math.abs(x)

Method Reference

Math::abs

Code Reuse

Existing methods can be reused without writing new lambdas.

Less Boilerplate

Instead of

```
s -> s.toUpperCase()
```

write

```
String::toUpperCase
```

Cleaner Streams

```
employees.stream()
```

```
    .map(Employee::getName)
```

```
    .forEach(System.out::println);
```

instead of

```
employees.stream()
```

```
    .map(e->e.getName())
```

```
    .forEach(name->System.out.println(name));
```

Lambda vs Method Reference

Lambda	Method Reference
s -> s.length()	String::length
x -> Math.abs(x)	Math::abs
(a,b) -> Integer.sum(a,b)	Integer::sum
() -> new ArrayList<>()	ArrayList::new
s -> System.out.println(s)	System.out::println

Anonymous Class vs Lambda vs Method Reference

Feature	Anonymous Class	Lambda	Method Reference
Java Version	1.1	8	8
Syntax	Long	Short	Shortest (when applicable)

Creates behavior	Yes	Yes	Refers to existing behavior
Needs existing method	No	No	Yes
Functional Interface	Yes	Yes	Yes

Summary

A method reference is **not** a direct method call. It is a **reference to an existing method** that Java adapts to the **single abstract method** of a compatible functional interface.

Internally, the process is:

1. The compiler identifies the target functional interface.
2. It checks that the referenced method's signature is compatible with the functional interface's abstract method.
3. The compiler emits an **invokedynamic** instruction instead of generating a traditional anonymous class.
4. At runtime, the JVM uses **LambdaMetafactory** to create an implementation of the functional interface that delegates to the referenced method.
5. When the functional interface method (`apply()`, `test()`, `accept()`, `get()`, etc.) is invoked, it forwards the call to the referenced method.

This design allows method references to be concise, readable, and efficiently optimized by the JVM while integrating seamlessly with lambdas, functional interfaces, and the Stream API.

Method Reference vs Lambda Expression in Java 8

A **lambda expression** and a **method reference** are both features introduced in **Java 8** to support **functional programming**. Both are used to provide an implementation of a **functional interface**, but they are not the same.

- A **lambda expression** defines **new behavior**.
- A **method reference** reuses **existing behavior**.

Definitions

Lambda Expression

A **lambda expression** is an anonymous function that provides an implementation for the **single abstract method (SAM)** of a functional interface.

Example:

```
Function<Integer, Integer> square = x -> x * x;
```

Here,

```
x -> x * x
```

is the lambda expression.

Method Reference

A **method reference** is a shorthand for a lambda expression that simply invokes an **existing method or constructor**.

Example:

```
Function<String, Integer> length = String::length;
```

Here,

```
String::length
```

is the method reference.

Lambda vs Method Reference

Feature	Lambda Expression	Method Reference
Introduced	Java 8	Java 8
Purpose	Define new behavior	Reuse an existing method
Syntax	parameter -> expression	ClassName::methodName
Creates new logic	✓ Yes	✗ No
Requires an existing method	✗ No	✓ Yes
Can contain multiple statements	✓ Yes	✗ No
Readability	Good	Often better when it only forwards to an existing method

Functional Interface Required	✓ Yes	✓ Yes
--------------------------------------	-------	-------

Example 1 – Function

Lambda

```
Function<String, Integer> length =
    s -> s.length();
```

```
System.out.println(length.apply("Java"));
```

Output

4

Method Reference

```
Function<String, Integer> length =
    String::length;
```

```
System.out.println(length.apply("Java"));
```

Output

4

Equivalent

```
s -> s.length()
```

↓

```
String::length
```

Example 2 – Consumer

Lambda

```
Consumer<String> printer =
    s -> System.out.println(s);
```

Method Reference

```
Consumer<String> printer =
    System.out::println;
```

Both

```
printer.accept("Hello");
```

Output

Hello

Example 3 – Predicate

Lambda

```
Predicate<String> empty =  
    s -> s.isEmpty();
```

Method Reference

```
Predicate<String> empty =  
    String::isEmpty;
```

Output

true

Example 4 – BinaryOperator

Lambda

```
BinaryOperator<Integer> add =  
    (a,b) -> a+b;
```

Method Reference

```
BinaryOperator<Integer> add =  
    Integer::sum;
```

Output

30

Example 5 – Supplier

Lambda

```
Supplier<List<String>> supplier =
```

```
() -> new ArrayList<>();
```

Method Reference

```
Supplier<List<String>> supplier =  
    ArrayList::new;
```

Stream API Example

Lambda

```
List<String> names =  
Arrays.asList("Java","Python","C");  
  
names.stream()  
    .map(s -> s.length())  
    .forEach(s -> System.out.println(s));
```

Method Reference

```
names.stream()  
    .map(String::length)  
    .forEach(System.out::println);
```

Output

```
4  
6  
1
```

Internal Working

Lambda Expression

```
Function<Integer,Integer> square =  
    x -> x*x;
```

Internally

Lambda

↓

Compiler

↓

invokedynamic



LambdaMetafactory



Function Object



apply()

The lambda contains **its own implementation**.

Method Reference

```
Function<String,Integer> length =  
    String::length;
```

Internally

Method Reference



Compiler



Checks compatibility



invokedynamic



LambdaMetafactory



Delegates to String.length()



apply()

The method reference **does not create new logic**. It simply delegates to an existing method.

Lambda Can Do More

Lambda

```
Function<Integer,Integer> fun =  
x -> {  
  
    int y = x*x;  
  
    y = y+100;  
  
    return y;  
  
};
```

Method Reference

Impossible.

There is no method reference equivalent because new logic is being created.

Method Reference Needs an Existing Method

Suppose

```
class MathUtil{  
  
    static int square(int x){  
  
        return x*x;  
  
    }  
  
}
```

Lambda

```
Function<Integer,Integer> fun =  
x -> MathUtil.square(x);
```

Method Reference

```
Function<Integer,Integer> fun =  
MathUtil::square;
```

Types of Method References

Type	Example	Equivalent Lambda
Static Method	Integer::parseInt	s -> Integer.parseInt(s)
Instance Method of Particular Object	scanner::nextLine	() -> scanner.nextLine()
Instance Method of Arbitrary Object	String::length	s -> s.length()
Constructor	ArrayList::new	() -> new ArrayList<>()

When Lambda Cannot Be Replaced

Lambda

```
Function<Integer,Integer> fun =  
x -> x*x + 100;
```

There is no existing method.

Cannot write

SomeClass::something

unless such a method already exists.

Another example

```
Comparator<Employee> cmp =  
(e1,e2)->{  
  
    if(e1.getSalary()>e2.getSalary())  
        return 1;  
  
    return -1;  
  
};
```

No method reference exists unless a suitable comparison method is already defined.

When Method Reference Is Better

Instead of

```
.map(e -> e.getName())
```

write

```
.map(Employee::getName)
```

Instead of

```
.forEach(x -> System.out.println(x))
```

write

```
.forEach(System.out::println)
```

Instead of

```
.filter(s -> s.isEmpty())
```

write

```
.filter(String::isEmpty)
```

Performance

Aspect	Lambda	Method Reference
Bytecode	invokedynamic	invokedynamic
JVM Optimization	Yes	Yes
Runtime Performance	Nearly identical	Nearly identical
Readability	Good	Often better when forwarding to an existing method

Performance differences are generally negligible. Choose based on clarity rather than speed.

Real-World Analogy

Imagine a restaurant.

Lambda

You tell the chef:

Take potatoes,
cut them,
fry them,
add salt,
serve.

You are describing the complete process.

Method Reference

Instead, you say:

Use Recipe #25.

The recipe already exists.

You simply reference it.

Decision Guide

Use a **lambda** when:

- You need to write new logic.
- Multiple statements are required.
- Conditions or calculations are involved.
- No existing method performs the required work.

Example:

$x \rightarrow x * x + 10$

Use a **method reference** when:

- An existing method already does exactly what you need.
- The lambda only forwards its arguments to another method.
- You want shorter, more readable code.

Example:

Math::abs

instead of

x -> Math.abs(x)

Summary

Aspect	Lambda Expression	Method Reference
Definition	Anonymous function that implements a functional interface	Reference to an existing method or constructor
Creates new behavior	✓ Yes	✗ No
Uses existing method	✗ Not required	✓ Required
Can contain loops/conditions	✓ Yes	✗ No
Supports multiple statements	✓ Yes	✗ No
Best for	Custom logic	Reusing existing methods
Example	x -> x * x	MathUtil::square
Stream Example	.map(x -> x.toUpperCase())	.map(String::toUpperCase)

Key Takeaways

- **Every method reference has an equivalent lambda expression.** For example:
- String::length

is equivalent to:

s -> s.length()

- **Not every lambda expression can be replaced by a method reference.** If the lambda contains custom logic, multiple statements, calculations, or branching, there may be no existing method to reference.
- Both lambdas and method references target **functional interfaces** and are implemented efficiently by the JVM using `invokedynamic` and `LambdaMetafactory`, so the choice between them should be driven primarily by **readability and maintainability**.